Programação Orientada a Objetos

Aula 02 - Introdução à POO com Python

Prof. Dr. Eduardo de Olivindo (eduardo.olivindo@ifce.edu.br)

Tecnologia em Análise e Desenvolvimento de Sistemas

IFCE - Campus Itapipoca




VIDADEPROGRAMADOR
.COM.BR
/* HISTÓRIA REAL ENVIADA POR WAGNER */
BY NC SA
#344
CARA... O QUE É "ORIENTADO A OBJETOS"?
OH, COFFEE MY COFFEE
É UMA FORMA DE PROGRAMAR QUE FOI CRIADA PARA TENTAR APROXIMAR O MUNDO REAL DO MUNDO VIRTUAL USANDO PROPRIEDADES, OBJETOS, MÉTODOS, ETC...
OH, COFFEE MY COFFEE
AHHHH TÁ... É QUE EU ACHO QUE FAÇO ORIENTADO A OBJETOS E NEM SEI...
PLAFT
OH, CO... MY COFFEE

Objetivos da aula

- Revisar objetos e métodos
- Criar classes próprias com construtor e atributos
- Definir métodos com regras e encapsulamento

Objetos e Métodos

Objetos e Métodos

Problema motivador

- Em programação procedural, dados e funções ficam separados
- Em sistemas maiores, isso gera:
  - dificuldade de manutenção
  - código espalhado
  - alto risco de inconsistências

Ideia da POO: juntar dados + comportamento em uma única estrutura: o objeto.

Objetos e Métodos

O que é um objeto?

- Um objeto representa uma entidade do mundo real ou do domínio
- Ele tem:
  - **estado** (atributos)
  - **comportamento** (métodos)

Exemplo: um relógio digital

- Estado: hora, minuto, segundo
- Comportamento: avançar segundo, ajustar hora, mostrar horário

Objetos e Métodos

Problema motivador

- Em Introdução à Programação, agrupamos dados com:
    - tuplas
    - dicionários
- Isso organiza informações, mas ainda não junta dados e comportamento específicas para aquela ideia de *coisa*.
- Contudo, tupo em Python é objeto, até as tuplas e dicionários.

Objetos e Métodos

Exemplo inicial: livro como *objeto* dicionário

- *Livro* deixa de ser uma *ideia* e passa a ser uma *coisa* na forma de dicionário.
- Um dicionário é um objeto próprio de seu tipo com seus dados e comportamentos.

```
nome = "Em busca da digitacao perdida"
autor = "Marcel Pythons"
ano = 1992
livro_dict = {"nome": nome, "autor": autor, "ano": ano}

print(livro_dict["nome"])
```

- Livro ainda não é um objeto com seus próprios dados e comportamentos.

Objetos e Métodos

O que é método?

- Métodos são funções ligadas ao objeto, descrevem seus comportamenteo.
- Nosso livro é um objeto do tipo dicionário que tem métodos próprios.
- Forma geral de acesso a métodos: `objeto.metodo(...)`

```
livro = {"nome": "O velho e as pythons", "autor": "Ernest Pythons", "ano": 1952}

for valor in livro.values(): # values() é um método de objetos dicionário
    print(valor)
```

- Livro ainda não é um objeto com seus próprios dados e comportamentos.

Objetos e Métodos

Tudo em python é objeto, mais exemplos.

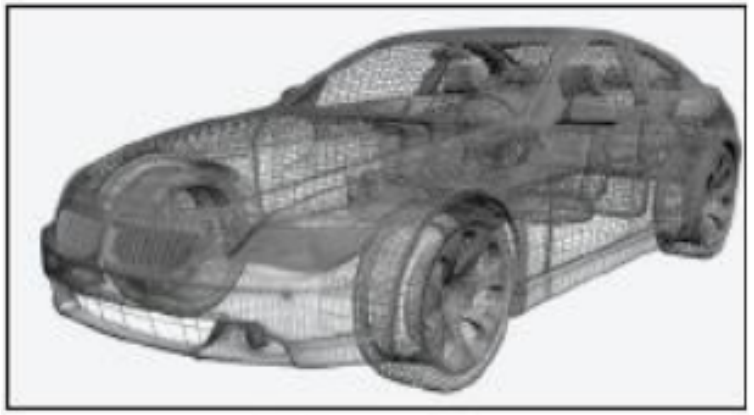
- Exemplos de métodos em objetos strings e listas:

```
nome = "Irene Imaginaria"
print(nome.count("I"))
print(nome.find("Irene"))
print(nome.replace("Irene", "Irineu"))
print(nome)

lista = [1, 2, 3]
lista.append(5)
lista.pop(0)
print(lista)
```

Classes e objetos

Uma classe é um modelo que define as atributos e os métodos comuns a todos os objetos do mesmo tipo.



Classe



Objeto

Classes e objetos

Criamos objetos a partir de um modelo inicial pré-definido

- A regra de criação de objetos é definida da definição da classe
- Chamamos esta regra de método **construtor** da classe.
- Executamos este método construtor para criar objetos:
- Sintaxe: `nome_da_variável_que_guarda_o_objeto = NomeDaClasse(argumentos)`

```
lista_1=list() # lista_1 = []
lista_2=list((1,2,3)) # lista_2=[1,2,3]
dict_1= dict(nome="Ana", idade=25) # dict_1={"nome":"Ana", "idade":25}
```

- Obs: objetos de tipos básicos podem ser criados com ambas sintaxes

Classes e objetos

Exemplo chamada de construtor

- Ao criar objetos de demais classes usamos o seu **construtor** conforme definido na documentação da classe.

```
from fractions import Fraction

metade = Fraction(1, 2)
terco = Fraction(1, 3)

print(metade)
print(terco)
print(metade + terco)
```

Classes e objetos



Classe é um molde, objeto é cada instância

- Cada objeto guarda seus próprios valores (atributos).

```
from datetime import date

vespera_natal = date(2020, 12, 24)
solsticio = date(2020, 6, 20)

print(vespera_natal.month) # Atributo mês
print(solsticio.month)
```

Classes e objetos

Objetos podem ser tratados como qualquer "variável"

- Exemplos de uso de objetos: Funções que recebem objetos

```
from datetime import date

def eh_fim_de_semana(data: date):
    dia_semana = data.isoweekday()
    return dia_semana == 6 or dia_semana == 7

print(eh_fim_de_semana(date(2020, 12, 24)))
print(eh_fim_de_semana(date(2020, 6, 20)))
```

Classes e objetos

Elementos do objeto: Método vs Atributo

```
from datetime import date

minha_data = date(2020, 12, 24)

dia_semana = minha_data.isoweekday()  # metodo
mes = minha_data.month                # atributo

print(dia_semana)
print(mes)
```

- Método usa parênteses
- Atributo não usa parênteses

Definindo Classes

Definindo Classes

A ideia é que definamos nossas próprias classes:

- Esqueleto de classe:

```
class ContaBancaria:
    pass
```

- A Primeira letra de cada palavra no nome da classe é maiúscula: *PascalCase*
- Criamos um objeto chamamndo o construtor padrão desta classe:

```
conta = ContaBancaria()
```

- Chamamos o ato de criar um objeto de **instanciar** um objeto.
- Um objeto também é chamado de uma **instância**.

Definindo Classes

Podemos adicionar atributos aos objetos

- Atributos podem ser criados fora de um construtor

```
class ContaBancaria:
    pass

conta_pedro = ContaBancaria()
conta_pedro.titular = "Pedro Python"
conta_pedro.saldo = 5.0

print(conta_pedro.titular)
print(conta_pedro.saldo)
```

- Funciona, mas é frágil.

Definindo Classes

Problema com a inclusão de atributos externos

```
class ContaBancaria:
    pass

conta_a = ContaBancaria()
conta_a.saldo = 100

conta_b = ContaBancaria()
print(conta_b.saldo)  # AttributeError
```

- Objetos diferentes podem nascer incompletos.

Definindo Classes

Construtor `__init__`

- Ao escrever a definição da função `__init__` estamos ditando as regras da criação de um objeto da classe.
- Definimos quais atributos devem ser passados como parâmetro de entrada e como serão salvos.

```
class ContaBancaria:
    def __init__(self, saldo: float, titular: str):
        self.saldo = saldo
        self.titular = titular

conta_pedro = ContaBancaria(100, "Pedro Python")
conta_paula = ContaBancaria(20000, "Paula Python")
```

Definindo Classes

Entendendo `self`

- o *self* representa a instância específica (o objeto real) que está sendo criada ou manipulada naquele momento.

```
class Livro:
    def __init__(self, nome: str, autor: str, genero: str, ano: int):
        self.nome = nome
        self.autor = autor
        self.genero = genero
        self.ano = ano
```

- `self.ano` é atributo do objeto
- `ano` é parâmetro local apenas passado como argumento

Definindo Classes

Usando objetos de suas classes

- Funções podem receber e alterar objetos

```
class ContaBancaria:
    def __init__(self, saldo: float, titular: str):
        self.saldo = saldo
        self.titular = titular

def abrir_conta(nome: str):
    return ContaBancaria(0, nome)

def depositar_em_conta(conta: ContaBancaria, valor: int):
    conta.saldo += valor
```

Exercício

1. Defina a classe `livro` com `nome`, `autor`, `genero` e `ano`
2. Escreva `livro_mais_antigo(livro1, livro2)`
3. Escreva `livros_por_genero(livros, genero)`

Definindo métodos

Definindo métodos

Classe com comportamento

- Métodos são os comportamentos das classes
- No nosso exemplo, um comportamento clássico de contas é depositar.
- A função depositar deve receber a instância sobre qual a operação será feita **self** e o valor a acrescentar do atributo **self.saldo**.

```
class ContaBancaria:
    def __init__(self,saldo: float, titular: str):
        self.titular = titular
        self.saldo = saldo

    def depositar(self,valor):
        self.saldo += valor
```

Definindo métodos

Método atua apenas no objeto-alvo

```
c1 = ContaBancaria(1000, "Pedro")
c2 = ContaBancaria(0, "Eduardo")
c3 = ContaBancaria(10000000000000000000000000000000000, "Silvio Santos")

c1.depositar(10)
c2.depositar(1)

print(c1.saldo)     # 1010
print(c2.saldo)     # 1
```

Definindo métodos

Encapsulamento na prática

- Atributos são alterados pelos métodos (encapsulamento dos atributos)

```
class ContaBancaria:
    def __init__(self, numero: str, titular: str, saldo: float, juros_anual: float):
        self.numero = numero
        self.titular = titular
        self.saldo = saldo
        self.juros_anual = juros_anual

    def sacar(self, valor: float):
        if valor <= self.saldo:
            self.saldo -= valor
            return True
        return False
```

- Cliente usa método. Regras ficam na classe.

Lista de Exercícios 01 - Casa

Outras considerações importantes

Importando uma classe

- É possível utilizar classes definidas em outros arquivos.
- `from carro import Carro` importa a classe `Carro` do módulo `carro`:

```
from carro import Carro # há um carro.py na mesma pasta

meu_carro = Carro("Fiat", "Uno")
print(meu_carro.modelo)
```

- Importando o arquivo completo :

```
import carro # há um carro.py na mesma pasta

meu_carro = carro.Carro("Fiat", "Uno")
print(meu_carro.modelo)
```

`if __name__ == "__main__"`

- `__name__` é uma variável especial do Python.
- Se o arquivo for executado diretamente, `__name__ == "__main__"`.
- Se o arquivo for apenas importado, esse bloco **não** é executado.

```python
def main():
    print("Programa principal em execução")

if __name__ == "__main__":
    main()
```

- Vantagens:
  - evita executar testes e exemplos ao importar o módulo
  - separa melhor definições de classes/funções da execução

Método `__str__`

- `__str__` é um método especial chamado por `print(objeto)` e `str(objeto)`.
- Serve para mostrar o objeto de forma legível para pessoas.
- Sem `__str__`, a impressão padrão costuma ser pouco informativa.

```
class Livro:
    def __init__(self, nome: str, autor: str, ano: int):
        self.nome = nome
        self.autor = autor
        self.ano = ano

    def __str__(self):
        return f"{self.nome} - {self.autor} ({self.ano})"
```

Método `__str__`

Teste:

```
if __name__ == "__main__":
    livro = Livro("Dom Casmurro", "Machado de Assis", 1899)
    print(livro)
```

Saída:

```
Dom Casmurro - Machado de Assis (1899)
```

- Vantagens:
  - melhora depuração e leitura da saída
  - deixa exemplos e relatórios mais claros
  - permite representar o objeto pelo que ele **significa**

Objetos e Referências

- Em Python, listas e variáveis normalmente não guardam uma cópia do objeto, mas uma **referência**.
- Assim, duas variáveis podem apontar para o mesmo objeto.
- A alteração feita em um objeto por uma referência aparece nas outras.

```
class Cachorro:
    def __init__(self, nome: str):
        self.nome = nome

dog1 = Cachorro("Totó")
dog2 = dog1

dog2.nome = "Lulu"
print(dog1.nome)  # Pooch
print(dog1 is dog2)  # True
```

Objetos e Referências

- E esse exemplo?

```
class Cachorro:
    def __init__(self, nome: str):
        self.nome = nome

dog1 = Cachorro("Totó")
dog2 = Cachorro("Totó")

dog2.nome = "Lulu"
print(dog1.nome)  # ????
print(dog1 is dog2)  # ???
```

Usar ou não o Self

- Use `self.atributo` quando a informação faz parte do **estado do objeto**.
- Use variáveis locais sem `self` (temporárias).

```
class Vocabulary:
    def __init__(self):
        self.words = []

    def longest_word(self):
        longest = ""  # não usar self.longest
        length_of_longest = 0 # não usar self.length_of_longest
        for word in self.words:
            if len(word) > length_of_longest:
                length_of_longest = len(word)
                longest = word
        return longest
```

Objeto como argumento de método

Um método pode receber outro objeto além de `self`

- `self` é o objeto sobre o qual o método foi chamado.
- Os demais parâmetros podem ser objetos de **qualquer** classe.
- Isso é comum em comparações, transferências e interações entre objetos.

Objeto como argumento de método

```
class ContaBancaria:
    def __init__(self, titular: str, saldo: float):
        self.titular = titular
        self.saldo = saldo

    def transferir(self, destino: "ContaBancaria", valor: float):
        if valor <= self.saldo:
            self.saldo -= valor
            destino.saldo += valor

conta_a = ContaBancaria("Ana", 100)
conta_b = ContaBancaria("Bruno", 50)
conta_a.transferir(conta_b, 30)
```

- No exemplo, `self` é a conta de origem.
- `destino` é outro objeto que participa da operação.

Lista de Exercícios 02 - Lab